Student Management System

Python-Based CRUD Application using OOP and File Handling

Project Type: Academic Python Project

Core Technologies: Python, Object-Oriented Programming, JSON File Handling

External Dependencies: None

Abstract

The Student Management System is a modular Python console application designed to efficiently maintain student records. It provides complete CRUD functionality, searching, filtering, persistent JSON storage, validation, and exception handling. The project demonstrates practical application of Object-Oriented Programming by separating student data, storage operations, business logic, and command-line interaction into dedicated modules.

1. Introduction

Managing student information manually can be time-consuming and error-prone. This project provides a simple digital solution that allows users to add, update, delete, search, filter, and view student records. The application is intentionally lightweight and uses JSON rather than a database, making it suitable for learning fundamental Python programming and software design concepts.

2. Objectives

The main objectives are to build a functional student record system, apply OOP principles, implement persistent file storage, provide reliable search and filtering, handle invalid input safely, and organize the source code into reusable modules.

3. Functional Requirements

Requirement	Implementation
Create	Add a new student after validating required fields.
Read	View all students or retrieve a student by ID.
Update	Modify selected fields of an existing student.
Delete	Remove a student record by ID.
Search	Search using ID, name, course, email, or phone.
Filter	Filter records by course.
Persistence	Save and load records from a JSON file.

Validation	Validate numeric age and basic email format.
Exception Handling	Handle duplicate records, missing records, invalid input and file/JSON errors.

4. System Architecture

The application follows a modular layered structure. The Student model represents the data entity. StudentStorage is responsible for JSON persistence. StudentManager contains the main CRUD, search, and filtering logic. Utility functions perform input validation and formatted output. The CLI module manages the menu and user interaction. main.py serves as the entry point.

5. Object-Oriented Programming

The Student class models an individual record using attributes such as student ID, name, age, course, email, and phone. StudentManager groups operations related to student records. StudentStorage encapsulates file operations. The separation of classes makes the system easier to understand, maintain, test, and extend.

6. File Handling

The system uses JSON for persistent storage. Every successful create, update, or delete operation writes the current student collection to data/students.json. When the application starts, existing records are loaded and reconstructed as Student objects.

7. Search and Filtering

Search is case-insensitive and checks multiple fields including student ID, name, course, email, and phone. Course filtering provides a focused way to display students belonging to a particular course.

8. Exception Handling

The application prevents common failures from terminating the program unexpectedly. Duplicate IDs and emails generate controlled errors, missing student records are reported, numeric fields are validated, and malformed or inaccessible JSON storage is handled safely.

9. Project Structure

```
main.py
student_management/models.py
student_management/storage.py
student_management/manager.py
student_management/utils.py
student_management/cli.py
data/students.json
```

10. Testing Plan

Test Case	Expected Result

Add unique student	Record is added and saved.
Add duplicate ID	Operation rejected with an error.
Update existing student	Selected fields are changed and saved.
Update missing student	Operation rejected.
Delete existing student	Record is removed.
Search by name	Matching records are displayed.
Filter by course	Only matching course records are displayed.
Invalid age	User is asked for a valid value.
Restart application	Previously saved records remain available.

11. How to Run

Install Python 3.9 or newer, open a terminal in the project folder, and execute `python main.py`. No external packages are required.

12. Conclusion

The completed project satisfies the required features of a Python-based Student Management System. It demonstrates CRUD operations, OOP, file handling, search and filtering, exception handling, and modular code organization. The design can later be extended with a graphical interface, database storage, authentication, reporting, and automated tests.

13. Future Scope

Potential improvements include replacing JSON with SQLite/MySQL, adding a Tkinter GUI, implementing login and role-based access, exporting records to CSV/PDF, adding sorting and pagination, and introducing automated unit tests.